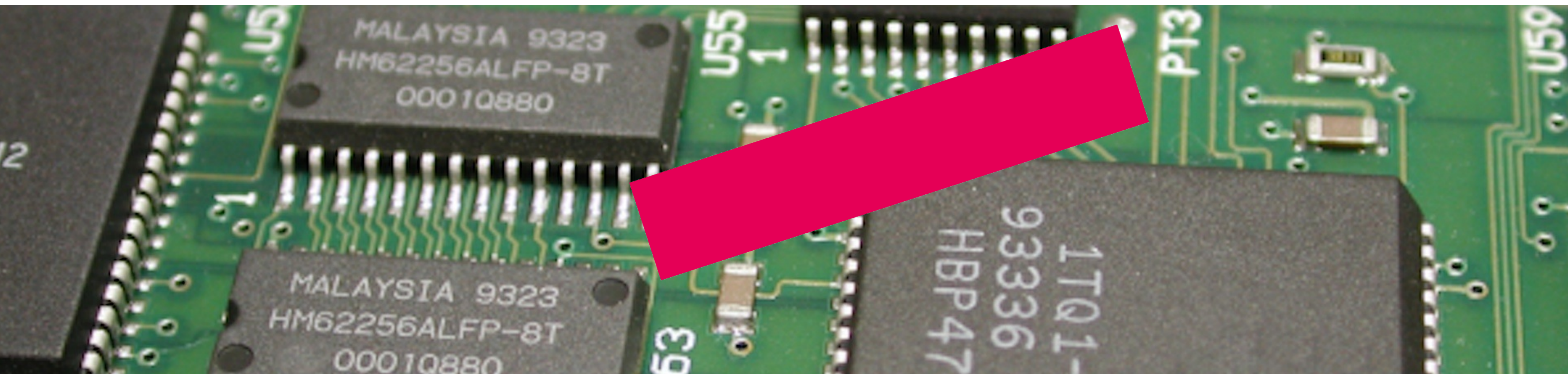


Prog 5 - 4. Interfaces and abstract classes



Electrical Engineering / Embedded Systems
Faculty of Engineering

Johan.Korten@han.nl

To begin with...

Attendance

Schedule (exact info see #00 and roster at insite.han.nl)

Week (Thu)	Design principle	C++	UML	Lab (Fri)
Step 1 - 3 Sep	Single Responsibility	Scope, namespaces, string	Use cases / class diagram	4 Sep: Git
Step 2 - 10 Sep	Open-Closed Principle	Constructors, iterators, lambdas	Inheritance / generalization	11 Sep: Requirements
Step 3 - 17 Sep	Liskov Substitution Principle	Lists, inline functions, default params	Activity diagrams	18 Sep: sensor library 1: core, Bus, Arduino
Step 4 - 24 Sep	Interface Segregation Principle	Interfaces and abstract classes	Dependencies	25 Sep: sensor library 2: Raspberry Pi, Bus / Clock split
Step 5 - 1 Oct	Dependency Inversion Principle	Threads, callbacks	Sequence diagrams	2 Oct: sensor library 3: sampler thread, callback, MQTT
Step 6 - 8 Oct	Coupling and cohesion	Polymorphism	Composition, packages	9 Oct: sensor library 4: polymorphism, UML design
Step 7 - 15 Oct	Embedded SOLID	Lifecycle: constructors, destructors, const	-	16 Oct: sensor library 5: RAIL, host unit tests, refactor

Note: subject to changes as we go...

Pure virtual in three rules

1. `virtual void f() = 0;` declares a function the base class does not implement. The `= 0` is syntax, not a value.
2. A class with at least one pure virtual function is abstract: you cannot create an instance of it, only of a subclass that implements every pure virtual function.
3. C++ has no interface keyword. An interface is an abstract class with only pure virtual functions and no data. Give it a virtual destructor.

That is all the language offers. Everything else today is about how you use it.

Last week you wrote this: the Bus interface

```
class Bus {
public:
    virtual ~Bus() = default;

    // Read `len` bytes starting at register `reg`.
    virtual bool read(uint8_t reg, uint8_t* data, size_t len) = 0;

    // Write `len` bytes starting at register `reg`.
    virtual bool write(uint8_t reg, const uint8_t* data, size_t len) = 0;

    // Block for at least `us` microseconds.
    virtual void delayUs(uint32_t us) = 0;
};

// Only <stdint> and <stddef>. No Wire.h, no Linux, no Arduino.
```

An interface is a contract: platform 1, Arduino

```
bool ArduinoI2cBus::read(uint8_t reg, uint8_t* data, size_t len) {  
    wire_.beginTransmission(address_);  
    wire_.write(reg);  
    if (wire_.endTransmission(false) != 0) return false;    // repeated start  
    if (wire_.requestFrom(address_, len) != len) return false;  
    for (size_t i = 0; i < len; ++i) data[i] = wire_.read();  
    return true;  
}
```

```
// The only file in the Arduino build that includes <Wire.h>.
```

Same contract: platform 2, Raspberry Pi

```
bool rpi::LinuxI2cBus::read(uint8_t reg, uint8_t* data, size_t len) {
    i2c_msg msgs[2] = {
        { address_, 0, 1, &reg }, // write register
        { address_, I2C_M_RD, uint16_t(len), data } // repeated start, read
    };
    i2c_rdwr_ioctl_data xfer{ msgs, 2 };
    return ::ioctl(fd_, I2C_RDWR, &xfer) >= 0;
}
```

```
// The only file in the Pi build that includes <linux/i2c-dev.h>.
// Bme280 did not change. Not one line.
```

Live demo: the same library on the Pi

On the Pi:

```
i2cdetect -y 1                # sensor at 0x76 or 0x77
cd Prog5/Code/week4/rpi_platform
cmake -B build && cmake --build build
./build/rpi_smoke_test
```

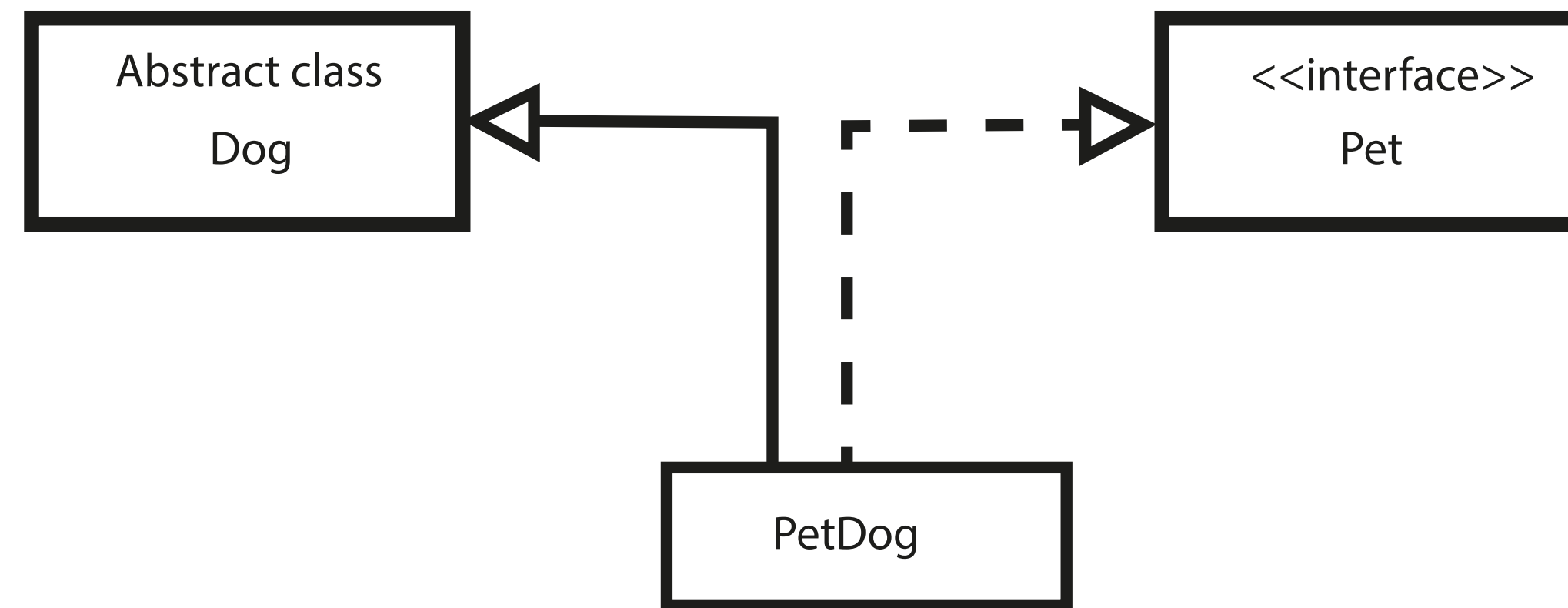
On the laptop:

```
cmake -B build && cmake --build build
-> core and tests build, rpi_smoke_test does not (Linux only)
```

That difference is the layering:

```
bmp280 (core) -> rpi_platform (Linux) -> rpi_smoke_test (app)
```

Abstract classes vs. Interfaces: UML



Inheritance: PetDog is-a Dog (Realization)



Interface: PetDog implements Pet (Generalisation)



Abstract classes vs. Interfaces: UML, on the sensor library

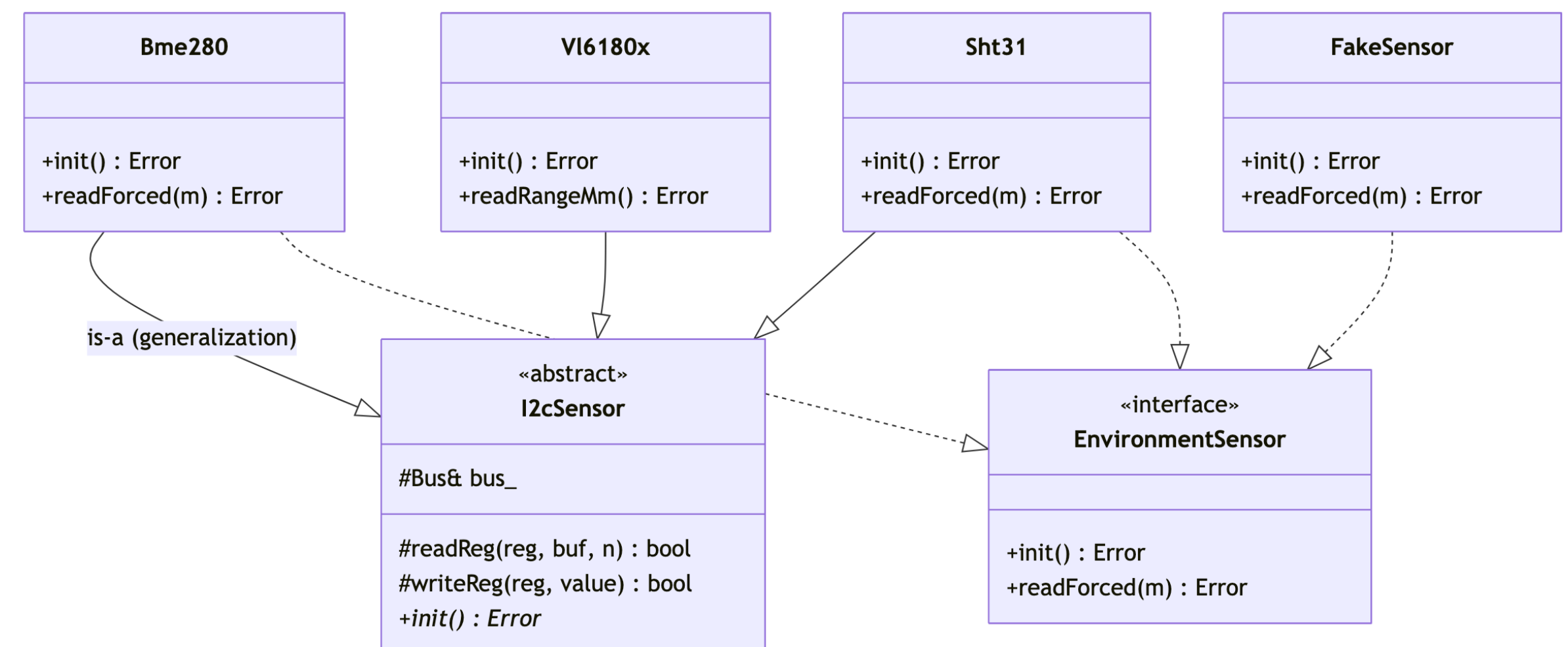
Interface: a role. Only pure virtual functions, no data.

EnvironmentSensor: "can be asked for a measurement".

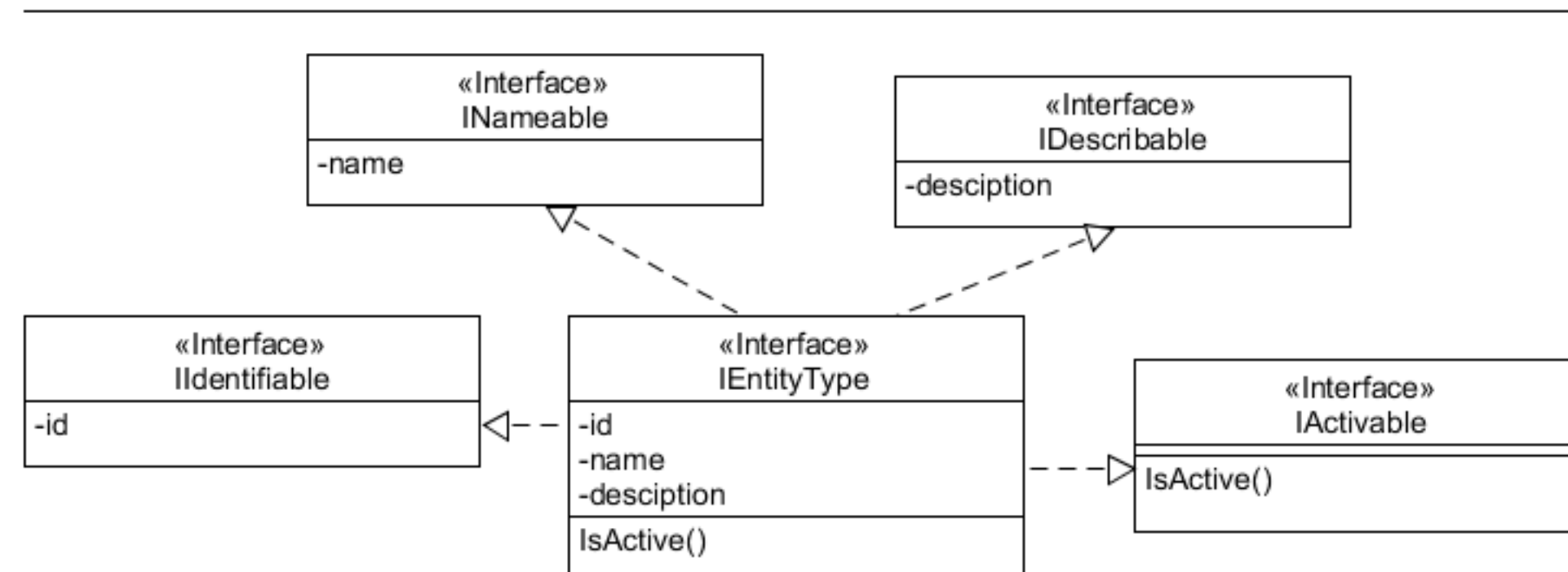
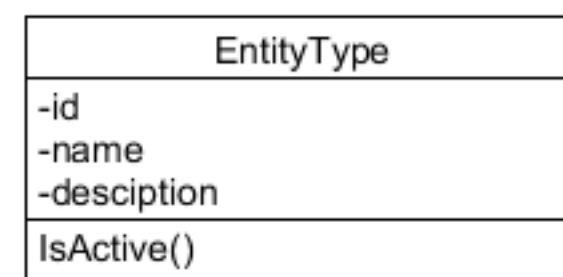
FakeSensor implements it without any I2C.

Abstract class: shared implementation plus at least one pure virtual. I2cSensor holds the Bus& and the readReg / writeReg helpers and leaves init() to the subclass. Bme280 is-a I2cSensor (generalization, solid arrow) and implements EnvironmentSensor (realization, dashed arrow).

For the sensor library we use interfaces: every seam we want to swap, mock or run on another platform (Bus, Clock, EnvironmentSensor). Shared helpers go into a small class you compose, not into a base class you inherit.



SOLID: Interface Segregation



"No client should be forced to depend on methods it does not use."

- Robert C. Martin

Make interfaces as slender as possible.

An interface that carries operations its clients never call is a fat interface: every implementation pays for it and every test double has to fake it. Next slide: we have one.

<https://claudiorivera.net/2018/02/04/isp-interface-segregation-principle/>

Interface Segregation, spotted

Look again at `Bus::delayUs()`.

A bus moves bytes. What does it have to do with time?

Who pays for it:

- MockBus in the tests must implement a delay it does not care about
- Arduinol2cBus and Linuxl2cBus copy the same three lines around different sleep calls
- a future SpiBus would copy them again

"No client should be forced to depend on methods it does not use."

Now the quote has a concrete victim.

The fix: Bus and Clock

```
class Bus {
public:
    virtual ~Bus() = default;
    virtual bool read(uint8_t reg, uint8_t* data, size_t len) = 0;
    virtual bool write(uint8_t reg, const uint8_t* data, size_t len) = 0;
};

class Clock {
public:
    virtual ~Clock() = default;
    virtual void delayUs(uint32_t us) = 0;
};

class Bme280 {
public:
    Bme280(Bus& bus, Clock& clock);    // two small contracts, not one fat one
    ...
};

// ArduinoClock, LinuxClock, FakeClock (tests: only counts the microseconds)
```

UML: dependencies. No arrow leaves the core

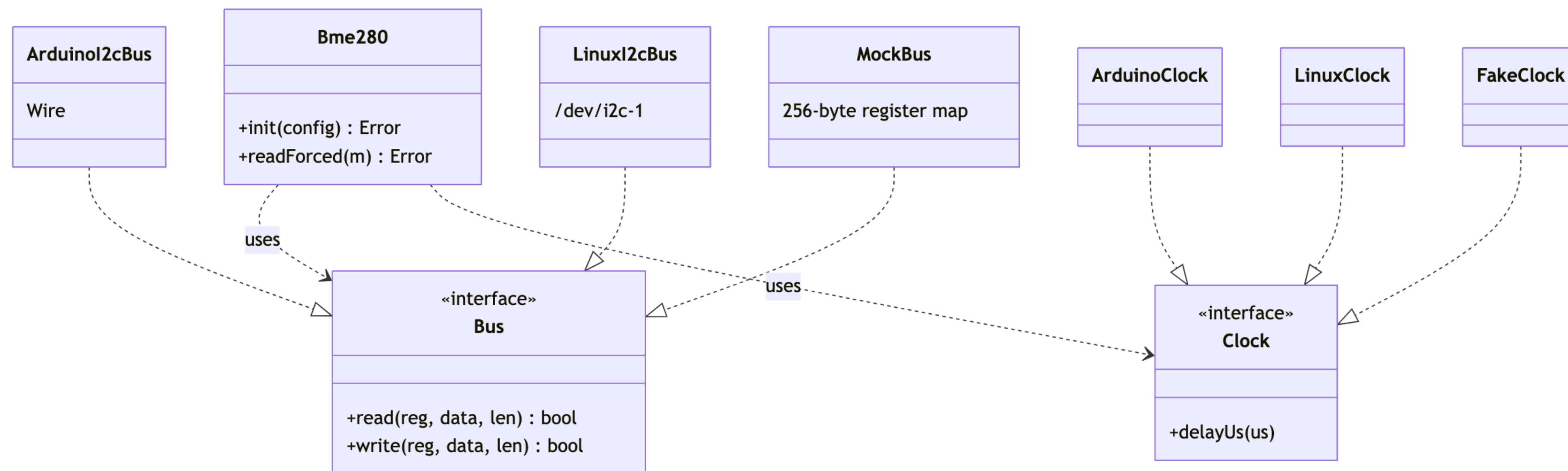
Realization (dashed, hollow head): implements

Dependency (dashed, open head): uses

Rule for your class diagram: not one arrow leaves the core towards a platform.

Try it: draw Wire and ioctl into this picture. They can only hang under the platform classes.

If you cannot draw it that way, the code has the problem, not the diagram.



Assignment 4: same library, second platform (deadline Thu 1 Oct, tag v0.4-pi)

1. Build the core on the Pi with CMake: three targets (Bosch C, wrapper, tests). Prove it builds on the Pi and on the laptop.
2. LinuxI2cBus: open in the constructor, close in the destructor, non-copyable.
3. Split Bus / Clock (ISP). ArduinoClock, LinuxClock, FakeClock. Write in the README what got simpler.
4. Smoke test on the Pi: one measurement per second, clean exit on Ctrl-C.
5. Class diagram with realizations and dependencies. No arrow leaves the core.

Layout: platform/arduino, platform/linux, examples/arduino, examples/rpi.

Full hand-out, starter code and tags: github.com/AEAEmbedded/ESE_PROG -> Prog5/Assignments

Links

- <https://www.codeproject.com/Articles/10553/Using-Interfaces-in-C>
- <https://data-flair.training/blogs/interfaces-in-cpp/>
- <https://www.tutorialcup.com/cplusplus/interfaces.htm>
- https://github.com/ksvbka/design_pattern_for_embedded_system/blob/master/design-patterns-for-embedded-systems-in-c-an-embedded-software-engineering-toolkit.pdf
- <https://martinfowler.com/articles/injection.html>

Try it yourself...

Cue: “What is the best way to use interfaces in C++?”

<https://copilot.microsoft.com/>

<https://gemini.google.com/>

<https://claude.ai/>

<https://chatgpt.com/>

Which one do you prefer?!

“

Any questions?